Mirpur Route Lab — Technical Report

1. Problem and objective This project investigates how graph-search algorithms choose routes in Mirpur, Dhaka, under different edge-cost assumptions. Real road topology is derived from OpenStreetMap (OSM) through the Overpass API. Users select start and destination points, alter synthetic congestion and safety-risk weights, mark roads blocked, and visualize search exploration. The target is a transparent demonstration of graph algorithms, not a real-world navigation or emergency-routing service.

2. Graph and assumptions The directed graph G=(V,E) contains OSM road vertices and consecutive road-segment links. Highway tags filter to drivable road categories; OSM oneway tags control reverse edges. User-selected geographical coordinates are snapped to the nearest retained OSM vertex, which may not be nearest by road travel. OSM mapping can be incomplete. Turn restrictions and road speed limits are not modeled. The server requests roads inside an approximate Mirpur bounding rectangle (23.778–23.840 N, 90.335–90.405 E). Graph edges use Haversine distance (km). The true graph size depends on the current external OSM response.

For edge (u,v), let d(u,v) denote physical segment length. A deterministic proxy risk q(u,v) and congestion c(u,v) are calculated from road category and stable endpoint identifiers. Generalized cost is C(u,v) = d(u,v) [1 + alpha*c(u,v) + beta*q(u,v)], where alpha and beta are nonnegative user inputs. In distance-only mode C(u,v)=d(u,v). This guarantees C(u,v)>=d(u,v), and edge costs remain nonnegative. Congestion and risk are simulated—not live or independently verified data. Blocks remove selected graph edges before search.

3. Search algorithms BFS uses FIFO queue and minimizes edge count. DFS uses a LIFO stack and is non-optimal. DLS limits depth; IDS repeats depth-limited passes from zero up to the specified maximum. Bidirectional BFS expands from the origin and a reverse-edge graph from destination to respect one-way roads. UCS prioritizes accumulated g(n), and A* prioritizes f(n)=g(n)+h(n). The heuristic h(n) is Haversine straight-line distance to destination and remains admissible for both cost modes because costs dominate distance. Weighted A* uses f(n)=g(n)+w*h(n), w>=1, trading guaranteed cost optimality for search focus; Greedy Best-First prioritizes h(n) alone. Beam Search ranks each depth's candidates by h(n) and retains a user-set width, which may discard viable branches.

Correctly completed UCS and A* searches return least-generalized-cost routes for the constructed graph; BFS and bidirectional BFS return minimum-edge-count paths. DLS, DFS, Greedy, Weighted A*, Beam, or depth-limited IDS need not minimize generalized cost. Weighted A* may have bounded suboptimality under standard assumptions, but this report makes no operational bound claim for truncated runs. Search uses a 50,000-expansion cap to protect the Vercel function; beyond that, a missing result is not proof a path does not exist.

4. Visualizations and measurements The browser displays OSM tiles, snapped route endpoints, final route, blocked-road selections, and expanded nodes sampled for rendering. Play/Pause/Next and a slider expose search order; the timeline replays **expansion order**, not a dynamic open/closed frontier animation. The dashboard gives path length in km, objective cost in generalized-distance units, expansion count, segment count, approximate maximum frontier size, and server-side `perf_counter` runtime in milliseconds. Time excludes browser rendering and network transport. An all-algorithms endpoint repeats the same search inputs for all 10 algorithms and provides CSV export.

5. 100/500 node experiment The tool chooses the first 100 or 500 road vertices ranked deterministically by sum of geographic distances to the selected endpoints, then includes the endpoints. It builds induced directed subgraphs retaining only links whose endpoints both belong to the selection. BFS, UCS, A*, and Greedy Search run on each graph. Graphs may be disconnected; null results represent absence of a connected route **within that induced graph. These sizes are educational controls, not repeated stochastic benchmarks or full asymptotic proofs. Compare expansions, cost and runtime. To obtain publication-quality results, run repeated trials, vary endpoints, separate cold vs warm API loads, and report mean/median with dispersion.**

6. Complexity and trade-offs With adjacency lists, BFS and DFS have worst-case time O(V+E) for standard graph traversal and storage O(V); the path-retaining teaching implementation may require more transient memory. UCS and A* with priority queues commonly take O((V+E) log V) under suitable heap behavior, subject to additional reopens and duplicates. DLS uses O(b*m) depth-path space in its textbook tree-search description, IDS repeated-search time is often O(b^d) for uniform branching, and bidirectional BFS can reduce breadth under suitable branching, though directed graph characteristics matter. Greedy and Beam may reduce expansions but sacrifice completeness/optimality with finite resource limits. Actual costs and times depend heavily on graph shape, edge weights, and parameter settings.

7. Validation, limitations and conclusion Local unit tests cover all algorithms' route reconstruction on a directed miniature graph, known shortest costs for UCS/A*, blockers, depth bounds, impossible reverse routes and identical endpoints. The live OSM Overpass endpoint and deployed Vercel instance must be tested separately; no live performance or quantitative advantage is claimed without measurements. The project demonstrates that optimized generalized-cost routing differs from minimum-edge routing, and that heuristic search trades search work against the possibility of suboptimal solutions. Real traffic, verified incident/safety data, and turn-by-turn navigation would require additional vetted data sources and engineering.